

MIP Formulations for a Rich Real-World Lot-Sizing Problem with Setup Carryover

Filippo Focacci¹, Fabio Furini^{2(✉)}, Virginie Gabrel², Daniel Godard¹,
and Xueying Shen^{1,2}

¹ DecisionBrain, 18 rue Yves Toudic, 75010 Paris, France

{ffocaci,daniel.godard,sylvia.shen}@decisionbrain.com

² PSL, CNRS, LAMSADE UMR 7243, Université Paris-Dauphine,
75775 Paris Cedex 16, France

{fabio.furini,virginie.gabrel}@dauphine.fr

Abstract. A rich lot-sizing problem is studied in this manuscript which comes from a real-world application. Our new lot-sizing problem combines several features, i.e., parallel machines, production time windows, backlogging, lost sale and setup carryover. Three mixed integer programming formulations are proposed. We theoretically and computationally compare these different formulations, testing them on real-world and randomly generated instances. Our study is the first step for efficiently tackling and solving this challenging real-world lot-sizing problem.

Keywords: Lot-sizing · Setup carryover · Mixed integer programming · Computational tests

1 Introduction

The *Lot-sizing Problem* (LSP) aims to plan the production in order to satisfy customer demands and to minimize operational costs. A number of different LSP variants have been studied in the literature developing *Mixed Integer Programming* (MIP) formulations. We refer the interested readers to [11] for a complete survey on the topic. In this manuscript, we study a rich real-world LSP variant presented in detail in the following.

Case Study. The problem is the core of a consulting project with an apparel company developed by DecisionBrain. The company runs multiple plants over Asia and provides services for many international brands. The production has to be planned in order to satisfy customer demands given limited resources. To produce one piece of clothes, a sequence of steps are required such as cutting, embroidery, sewing and washing. We concentrate on the bottleneck of the production process, i.e., the sewing process. This is the only step executed by workers instead of machines. A product can be executed by any production line consisting of a team of workers. When a production line switches production

from one product to another, a setup adjustment occurs which consumes capacities and generates costs. In this problem, each demand consists of a quantity of one product to be produced between its release date and due date. The release date is due to material availability and the production for this demand can only start from this date. Before the due date, the demand can be satisfied at any time without extra cost. After the due date, the demand can still be satisfied causing an additional tardiness cost. Two levels of tardiness costs are considered: the first is due to a transport cost (an air transport instead of a maritime one necessary to meet the due date), while the second is due to a discount offered to the customer for compensation. Moreover, the unsatisfaction of demands is also allowed with a very high penalty cost. The goal of the company is to plan its production in order to minimize the cost. Considering that the production lines are nearly fully loaded, an efficient production planning is crucial to increase company competitiveness. This project focuses on medium-term planning decisions, in which the planning horizon is divided into a set of consecutive time buckets, such as weeks. The medium-term planning decides the production quantities in each time bucket for each production line. This problem can be modeled as a capacitated lot-sizing problem with setup carryover, parallel machines, production time windows, backlogging and lost sale.

Main Problem Features. Any product can be produced on any production line, called machine for brevity in the following. The assigned machine must be tuned at a state according to the product, which is called a *setup state* for the product. When the machine switches from one setup state to another within a time bucket, a setup occurs which causes a setup capacity and a setup cost. The last setup state of a time bucket is carried over to the beginning of the next time bucket, which is known as setup carryover [4]. Therefore, if the last setup state of $t - 1$ is for product i , then producing i at the beginning of t does not cause any setup cost or setup capacity. Setting up the machines for the production of a product is only allowed once for each time bucket. The LSP with setup carryover has been first studied in [4], in which a MIP model has been proposed and a fix-and-relax heuristic algorithm has been developed. Different MIP formulations have been also proposed, see [6–8, 13, 14].

Another major difficulties of our problem comes from *parallel machines*. Based on the application, we consider uniform parallel machines, in which the machines have independent capacities and the consumed capacity for each product only depends on the product itself. The introduction of parallel machines may lead to a large amount of symmetric solutions, therefore it increases the difficulties of the problem. In our application, there are up to 29 parallel machines with different capacities. In [15] two branch and bound algorithms are developed to solve the LSP with parallel machines. Also the Lagrangian relaxation is a widely used technique to address this problem [5, 16].

In the LSP with *Production Time Window* (PTW), the production for each demand can only happen between its release time and due date. The release time represents several situations such as raw material available dates. The LSP with

PTW has been first addressed by [3]. Polyhedral studies and exact algorithms have been developed in [2, 3, 9, 17].

Backlogging and *Lost sale* (BL) (see [12, 18]) are also considered in our problem, which means that the demands can be satisfied later than their due dates or even not satisfied incurring in a penalty cost. These are quite common features in real-world applications when the production capacity is insufficient to satisfy all the demands. The LSP with PTW and BL has been studied in [1], for the case of single product and an uncapacitated machine.

Problem Definition. The input parameters of our problem are:

- $\mathcal{T} = \{1, 2, \dots, T\}$: set of time buckets.
- $\mathcal{M} = \{1, 2, \dots, M\}$: set of machines.
- $\mathcal{N} = \{1, 2, \dots, N\}$: set of products.
- $\mathcal{D} = \{1, 2, \dots, D\}$: set of demands.
- cap_{rt} : capacity of machine r in time bucket t ($r \in \mathcal{M}, t \in \mathcal{T}$).
- cap_i : capacity required by unitary production of product i ($i \in \mathcal{N}$).
- st_{ir} : setup capacity for product i on machine r ($i \in \mathcal{N}, r \in \mathcal{M}$).
- sc_{ir} : setup cost for product i on machine r ($i \in \mathcal{N}, r \in \mathcal{M}$).
- $p_d \in \mathcal{N}$: the required product of the demand d ($d \in \mathcal{D}$).
- q_d : quantity of product p_d required by demand d ($d \in \mathcal{D}$).
- b_d : release date of demand d ($d \in \mathcal{D}$).
- e_d^1 : first due date of demand d ($d \in \mathcal{D}$). No extra cost in interval $[b_d, e_d^1]$.
- e_d^2 : second due date of demand d ($d \in \mathcal{D}$).
- tc_d^1 : unitary extra cost for demand d satisfied at or after e_d^1 ($d \in \mathcal{D}$).
- tc_d^2 : unitary extra cost for demand d satisfied at or after e_d^2 ($d \in \mathcal{D}$).
- lc_d : unitary cost for unsatisfied demand d ($d \in \mathcal{D}, lc_d > tc_d^1 + tc_d^2$).

The problem is to decide for each machine $r \in \mathcal{M}$ and for each time bucket $t \in \mathcal{T}$, how much to produce of each product $i \in \mathcal{N}$. The objective is to minimize the total cost including lost sale cost, tardiness cost and setup cost without exceeding the machine capacities cap_{rt} for each machine r and time bucket t ($r \in \mathcal{M}, t \in \mathcal{T}$). Note that the production is used to satisfy demands directly so there are no inventory costs.

To the best of our knowledge, it is the first time that such a rich LSP is studied combining many of the important real-world features.

2 MIP Formulations

Different formulations concerning setup carryover have been proposed in the literature. In this section, we extend 3 MIP formulations for the LSP defined above. These formulations mainly differ in the way the setup carryover is modelled. They all share a common part concerning the Lost Sale costs, the Tardiness cost and material flow conservation constraints.

For each product $i \in \mathcal{N}$, each machine $r \in \mathcal{M}$, each time bucket $t \in \mathcal{T}$ and each demand $d \in \mathcal{D}$, we introduce the following decision variables:

- $x_{irt} \in \mathbb{R}^+$: the production quantity of product i on machine r during time t .
- $y_{dt} \in [0, q_d]$: the satisfied quantity of demand d in time bucket $t \geq b_d$.
- $y_d \in [0, q_d]$: the unsatisfied quantity of demand d .

The lost Lost Sale and Tardiness cost are defined as follows:

$$LostSaleCost = \sum_{d \in \mathcal{D}} lc_d y_d, \quad (1)$$

$$TardinessCost = \sum_{d \in \mathcal{D}, t \in \mathcal{T}: t \geq e_d^1} tc_d^1 y_{dt} + \sum_{d \in \mathcal{D}, t \in \mathcal{T}: t \geq e_d^2} tc_d^2 y_{dt}, \quad (2)$$

The constraints modelling the demand satisfaction can be written as follows:

$$\sum_{r \in \mathcal{M}} x_{irt} = \sum_{d \in \mathcal{D}: p_d = i} y_{dt} \quad i \in \mathcal{N}, t \in \mathcal{T} \quad (3)$$

$$\sum_{t \in \mathcal{T}} y_{dt} + y_d = q_d \quad d \in \mathcal{D} \quad (4)$$

$$0 \leq x_{irt} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (5)$$

$$0 \leq y_{dt}, y_d \leq q_d \quad d \in \mathcal{D}, t \geq b_d \quad (6)$$

The material flow conservation is formulated as (3). Constraints (4) guarantee that for each demand, the summation of the satisfied amount and the unsatisfied amount equals to the required quantity. Finally, due to the release date, y_{dt} is only defined from the release time b_d for each demand d in (6).

If no setup is considered, the problem is a linear programming problem, which can be solved in polynomial time. Therefore, the complexity mainly comes from the setup. The machine capacity is both consumed by production capacity $ProdCap(r, t)$ and setup capacity. The capacity consumption due to production can be expressed as follows:

$$ProdCap(r, t) = \sum_{i \in \mathcal{N}} cap_i x_{irt} \quad r \in \mathcal{M}, t \in \mathcal{T}. \quad (7)$$

In the next sections, we present 3 different ways to model the setup cost and setup capacity consumption.

2.1 Formulation 1

In Haase [8], a MIP formulation for the LSP on a single machine with setup carryover has been introduced. We adapt this formulation to our problem and introduce setup variables for each product $i \in \mathcal{N}$, each machine $r \in \mathcal{M}$ and each time bucket $t \in \mathcal{T}$ as follows:

- $v_{rt} \in [0, 1]$, $v_{rt} > 0$ indicates if more than one product is produced in time bucket t and for machine r .
- $z_{irt} \in \{0, 1\}$ equals to 1 if a setup state for product i on machine r exists in time bucket t and 0 otherwise.

– $z_{irt}^c \in \{0, 1\}$ equals to 1 if the setup state for product i is carried over from time bucket $t - 1$ to time bucket t on machine r and 0 otherwise.

Then the first formulation (*Form1*) is formally given as follows ($\tilde{\mathcal{T}} = \mathcal{T} \setminus \{1\}$):

$$\min \quad (1) + (2) + \sum_{i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T}} sc_{ir}(z_{irt} - z_{irt}^c) \quad (8)$$

$$s.t. \quad (3) - (6)$$

$$ProdCap(r, t) + \sum_{i \in \mathcal{N}} st_{ir}(z_{irt} - z_{irt}^c) \leq cap_{rt} \quad r \in \mathcal{M}, t \in \mathcal{T} \quad (9)$$

$$x_{irt} \leq \Theta_{irt} z_{irt} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (10)$$

$$\sum_{i \in \mathcal{N}} z_{irt}^c \leq 1 \quad r \in \mathcal{M}, t \in \mathcal{T} \quad (11)$$

$$z_{irt}^c \leq z_{ir, t-1} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \tilde{\mathcal{T}} \quad (12)$$

$$z_{irt}^c \leq z_{irt} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (13)$$

$$z_{irt}^c + z_{ir, t-1}^c + v_{r, t-1} \leq 2 \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \tilde{\mathcal{T}} \quad (14)$$

$$Nv_{rt} - \sum_{i \in \mathcal{N}} z_{irt} + 1 \geq 0 \quad r \in \mathcal{M}, t \in \mathcal{T} \quad (15)$$

$$z_{irt}^c, z_{irt} \in \{0, 1\} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (16)$$

$$v_{rt} \in [0, 1] \quad r \in \mathcal{M}, t \in \mathcal{T} \quad (17)$$

where Θ_{irt} is a large enough constant that never unnecessarily limits the production x_{irt} .

According to the definition of setup carryover, a setup cost has to be paid when there is a setup ($z_{irt} = 1$) which is not carried over from the last time bucket ($z_{irt}^c = 0$). Constraints (9) ensure that the capacity is not exceeded on each machine in each time bucket, where the setup capacity consumption is formulated similarly to the setup cost. Constraints (10) link the production and the setup since a positive production of i on r at t requires a setup state for i on r at t . For the setup carryover, there is at most one setup state to be carried over to the next time bucket, which is guaranteed by constraints (11). A setup state of i on r carried over from $t - 1$ to t implies that this state is included in both $t - 1$ (12) and t (13). If there is more than one setup state in one time bucket, i.e., $v_{rt} > 0$, the initial setup state and the last setup state are necessarily different. This is formulated as constraints (14). Finally, to fulfill the definition of variable v_{rt} , we have the constraints (15) ($N = |\mathcal{N}|$, i.e., the number of different products).

2.2 Formulation 2

In Sox et al. [13], several MIP formulations for the LSP on single machine with setup carryover are presented. We adapt one of them to our problem introducing the following setup variables for each product $i \in \mathcal{N}$, each machine $r \in \mathcal{M}$ and each time bucket $t \in \mathcal{T}$:

- $z_{irt}^0 \in \{0, 1\}$ equals to 1 if the initial setup state is for product i on machine r in time bucket t , implying that the final setup state for $t - 1$ on r is for product i .
- $z_{irt}^+ \in \{0, 1\}$ equals to 1 if there is a state switch for product i on machine r in time bucket t .

Then the second formulation (*Form2*) is formally given as follows:

$$\min \quad (1) + (2) + \sum_{i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T}} sc_{ir} z_{irt}^+ \quad (18)$$

$$s.t. \quad (3) - (6)$$

$$ProdCap(r, t) + \sum_{i \in \mathcal{N}} st_{ir} z_{irt}^+ \leq cap_{rt} \quad r \in \mathcal{M}, t \in \mathcal{T} \quad (19)$$

$$x_{irt} \leq \Theta_{irt}(z_{irt}^0 + z_{irt}^+) \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (20)$$

$$\sum_{i \in \mathcal{N}} z_{irt}^0 = 1 \quad r \in \mathcal{M}, t \in \mathcal{T} \quad (21)$$

$$z_{irt}^0 \leq z_{ir, t-1}^0 + z_{ir, t-1}^+ \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \tilde{\mathcal{T}} \quad (22)$$

$$z_{jr, t-1}^+ \leq 2 - z_{ir, t-1}^0 - z_{irt}^0 \quad i, j \neq i \in \mathcal{N}, r \in \mathcal{M}, t \in \tilde{\mathcal{T}} \quad (23)$$

$$z_{irt}^0, z_{irt}^+ \in \{0, 1\} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (24)$$

The total setup cost is formulated as $\sum_{i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T}} sc_{ir} z_{irt}^+$ since the setup cost has to be paid only when there is a setup switch ($z_{irt}^+ = 1$). Constraints (19) ensure that the total used capacity does not exceed the available capacity, where the setup capacity consumption is formulated similarly to the setup cost. Constraints (20) link the setup and production since a positive production of i on r at t requires a setup state for i on r at t , which is either from an initial setup state or a setup switch. There is a unique initial setup state for each time bucket on each machine, which is established by constraints (21). Also, the initial setup state must be one of the setup states in the previous time bucket (22). However, constraints (23) ensure that, on machine r during time bucket t , no setup switch is possible when the initial setup state and the last setup state of t (i.e., the initial setup state of the next time bucket $t + 1$) are both for the same product.

2.3 Formulation 3

In Suerie et al. [14], a MIP formulation for the LSP on single or multi-level with setup carryover is presented. This formulation is similar to the *Form2*. In addition to the previously defined binary variables $z_{irt}^0 \in \{0, 1\}$ and $z_{irt}^+ \in \{0, 1\}$ for each product $i \in \mathcal{N}$, for each machine $r \in \mathcal{M}$, each time bucket $t \in \mathcal{T}$, additional variables $w_{rt} \in \{0, 1\}$ are introduced. w_{rt} equals to 1 if only one product is produced on r in t , and 0 otherwise. Then the third formulation (*Form3*) is formally given as follows:

$$\min \quad (1) + (2) + \sum_{i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T}} sc_{ir} z_{irt}^+ \quad (25)$$

$$s.t. \quad (3) - (6), (19) - (22), (24)$$

$$z_{irt}^0 + z_{ir,t-1}^0 \leq 1 + w_{r,t-1} \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \tilde{\mathcal{T}} \quad (26)$$

$$z_{irt}^+ + w_{rt} \leq 1 \quad i \in \mathcal{N}, r \in \mathcal{M}, t \in \mathcal{T} \quad (27)$$

$$0 \leq w_{rt} \leq 1 \quad \forall r \in \mathcal{M}, t \in \mathcal{T} \quad (28)$$

Constraints (26) implies that on machine r , the initial setup states at $t - 1$ and t have to be different when more than one product is produced during $t - 1$ ($w_{r,t-1} = 0$). Constraints (27) ensures that $w_{rt} = 0$ when there are more than one setup state during time bucket t . Thanks to the other constraints of the formulation, it is not necessary to impose integrality constraints for the w_{rt} variables. Due to constraints (21) and (22), we have $z_{ir,t+1}^0 = z_{irt}^0 = 1$, which forces $w_{rt} = 1$.

3 Theoretical Comparison

In this section, we compare the previously introduced MIP formulations strength in terms of lower bounds given by the Linear Programming (LP) relaxation.

Let $(\tilde{x}, \tilde{y}, \tilde{z}^c, \tilde{z}, \tilde{v})$, $(\bar{x}, \bar{y}, \bar{z}^0, \bar{z}^+)$ and $(\dot{x}, \dot{y}, \dot{z}^0, \dot{z}^+, \dot{w})$ be optimal solutions of the LP relaxation of *Form1*, *Form2* and *Form3* respectively, while $f_1(\tilde{x}, \tilde{y}, \tilde{z}^c, \tilde{z}, \tilde{v})$, $f_2(\bar{x}, \bar{y}, \bar{z}^0, \bar{z}^+)$ and $f_3(\dot{x}, \dot{y}, \dot{z}^0, \dot{z}^+, \dot{w})$ are the corresponding optimal objective function values. Then the following theorem holds:

Theorem 1

$$f_2(\bar{x}, \bar{y}, \bar{z}^0, \bar{z}^+) = f_3(\dot{x}, \dot{y}, \dot{z}^0, \dot{z}^+, \dot{w}) \geq f_1(\tilde{x}, \tilde{y}, \tilde{z}^c, \tilde{z}, \tilde{v}) .$$

This theorem shows that the LP relaxation of *Form2* and *Form3* provide equivalently better lower bounds than *Form1*. The proof of the theorem requires a number of technicalities and for reason of space we decided to omit the proof.

A comparison of the formulation size is summarized in the Table 1. They all have the same number of binary variables, while *Form2* has less continuous variables than the other two. The number of constraints increases in the order *Form3*, *Form1* and *Form2*.

Table 1. Formulation size comparison

Form	Number of variables	# Binary	Number of constraints
Form1	$3NMT + MT + DT + D$	$2NMT$	$NT + MT + 3NMT - 2NM + \Pi$
Form2	$3NMT + DT + D$	$2NMT$	$MT + NM(T - 1) + \frac{N(N-1)}{2}M(T - 1) + \Pi$
Form3	$3NMT + MT + DT + D$	$2NMT$	$MT + 2NMT - NM + \Pi$
$\Pi := NMT + NT + MT + D$			

4 Computational Comparison

In this section, we test the performances of the three MIP formulations, by comparing the computational difficulties to find optimal solutions using CPLEX 12.6 as a MIP solver.

The testbed consists of 3 real-world instances and 810 randomly generated instances. The 3 real-world instances have the following features: $T = 27, 36, 30$, $M = 3, 28, 29$, $N = 3, 18, 1$, $D = 313, 1188, 595$, and $R = 0.99, 0.14, 0.33$, where R is a indicator representing the capacity usage defined as $R = \frac{\sum_{d \in \mathcal{D}} cap_{p_d} q_d}{\sum_{r \in \mathcal{M}, t \in \mathcal{T}} cap_{rt}}$. Although R does not consider the setup capacity, it is an indicator of the machine loads. For the randomly generated instances, we consider the following parameters: $T \in \{4, 9, 13\}$, $M \in \{1, 5, 10\}$, $N \in \{4, 8, 12\}$, $D \in \{50, 100, 200\}$ and $R \approx \{0.75, 0.9\}$. Assuming that one time bucket corresponds to one week, we choose the number of time buckets from one month ($T = 4$) to a season ($T = 13$). The other parameters are chosen to generate instances with different levels of difficulty. For each possible combination of the parameters, we generate five different instances to limit bias. All the experiments run on one core of an Intel Core i7-4790 2.50 GHz 3.60, with 16 GB shared memory, under the Linux Ubuntu 12.4 operating system.

In the Table 2, we present the computational results using CPLEX to solve the MIP and LP models on the benchmark instances setting a time limit of 10 min. In the table, the computing time is expressed in seconds. The results are presented for the randomly generated instances in the upper part of the table and for the 3 real-world instances in the bottom part. For the randomly generated instances, we give the average results for each value of the parameter T , M , D , N and R . Averages values over all the randomly generated instances are reported in the Row TOT/AVG. The Column #Opt reports the total number of optimally solved instances for each formulation. In the first two columns, we present the parameters and their values. In Column LPGap, we measure the quality of the LP relaxation by showing the average LPGap, which is calculated as $\frac{BestMip - LPVal}{BestMip}$, where *bestMip* is the best known MIP solution for the instance and *LPVal* is the optimal LP relaxation objective value. On all instances except two, the three formulations are characterized by the same *LPVal*. In the remaining two cases the difference is very small, thus we only report the LPGap once in the table. The value of Θ_{irt} has been set to $\Theta_{irt} = \min \left\{ \frac{cap_{rt}}{cap_i}, \sum_{d: p_d=i, t \geq b_d} q_d \right\}$. In Column #Opt and MipTime, we report the number of instances solved to prove optimality within the time limit of 10 min and the average computing time. In Column #Nodes and Gap, we report the number of explored nodes and, for the instances not solved to optimality within the time limit, we report the exit gap. This gap represents the relative difference between the primal and the dual bounds computed by the CPLEX at the time limit. For the real instances, we report the objective function values returned by the solver in Column MipObj.

As far as the computing time necessary to calculate the LP relaxation is concerned, the average values over all the instances are 0.37 s, 0.74 s and 0.30 s for *Form1*, *Form2* and *Form3* respectively. Thanks to the shortest average

computing time for the LP relaxation, *Form3* is able to explore more nodes within the given time limit. As expected, according to the Column #Opt, the table shows that as the problem size increases, the instances become more difficult to solve. The parameter N , which has an impact on the number of binary variables, affects the most the solvability of the instances. Take for example *Form2*, when N increases from 4 to 12, the number of instances decreases from 258 to 59, whereas when D increases from 50 to 200, the number is only reduced by 36. The number of time buckets T has a smaller impact on the computing time. We can observe that *Form1* explores a higher number of nodes, this is probably due to the fact that it struggles to find good quality integer solutions. On the other hand, *Form2* and *Form3* explore almost the same number of nodes for the randomly generated instances, while the *Form3* explores more nodes for the 3 real-world instances. Regarding the number of randomly generated instances solved to be proven optimality, *Form3* solves 416 instances and *Form2* solves 411 instances, while *Form1* solves only 379. A similar behavior can be also observed for the exit gap. For the real-world instances, we observe similar results, i.e., the *Form3* shows the best performance. Hence, according to the computational experiments, *Form3* shows the best overall computational performance. This is due to the fact that it has the least number of constraints with the same number of binary variables compared to the other two formulations, and its LP relaxation can be solved faster.

We also compare the performances of the three formulations using the Performance Profile [10] in Fig. 1. Let $t_{p,s}$, $o_{p,s}$ be the computing time and the objective function value of instance $p \in P$, where P is the set of instances, given by formulation $s \in S = \{\textit{Form1}, \textit{Form2}, \textit{Form3}\}$. Let b_p equal to $\min_{s \in S} \{o_{p,s}\}$. The performance ratio $\rho_s(\tau)$ of a given $\tau \geq 1$ is defined as follows:

$$\rho_s(\tau) = \frac{1}{|P|} \sum_{p \in P, r_{p,s} \leq \tau} 1, \quad r_{p,s} = \begin{cases} \frac{t_{p,s}}{\min_{s \in S} \{t_{p,s}\}} & \text{if } |\frac{o_{p,s} - b_p}{b_p}| \leq \delta, \\ \rho_M & \text{otherwise,} \end{cases}$$

where ρ_M is a large enough number such that $\rho_M > r_{p,s}$ for $p \in P$ and $s \in S$. When $\delta = 0$, the ratio $\rho_s(\tau)$ represents the percentage of instances for which a given formulation s returns the best known solutions given time $\tau \min_{s \in S} \{t_{p,s}\}$ for each instance. Due to the numerical precision of CPLEX, we set $\delta = 10^{-6}$. In the Fig. 1, the horizontal axis represents the “Time Factor τ ” using the logarithmic scale. The vertical axis represents the performance ratio $\rho_s(\tau)$. When $\tau = 1$, the performance ratio gives the percentage of instances that are solved fastest and best by each formulation. By “best” we mean computing the best known objective function value. We observe that *Form3* obtains the best known solution in shortest computing time for approximately 42 % instances, 36 % for *Form2* and 30 % for *Form1*. Moreover, when τ increase, the performance ratio tends to the percentage of instances that are solved best by each formulation within the time limit. *Form3* obtains the best solutions for approximately 78 % instances. Therefore, we can conclude that *Form3* gives the overall best performance.

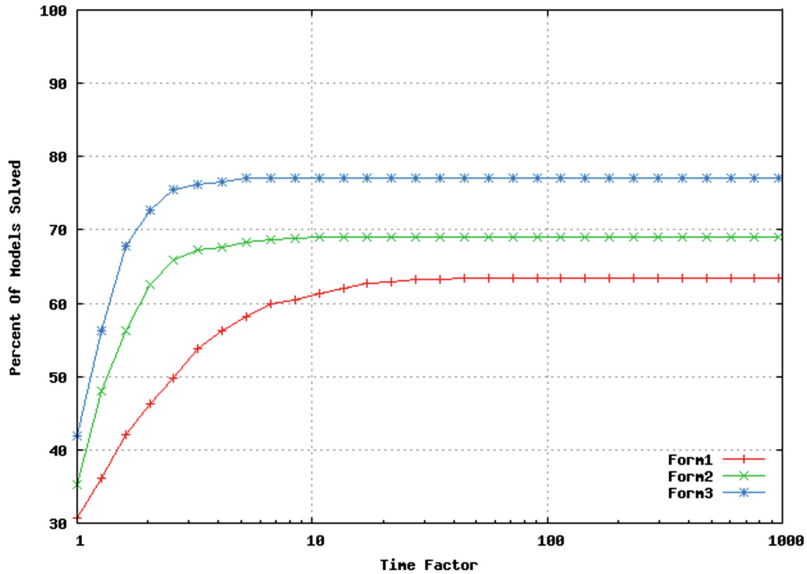


Fig. 1. Performance profile

5 Conclusion

We study a new variant of the LSP, which is based on a real-world application. The problem combines, for the first time, several classical features of the LSP such as setup carryover and PTW. We present and compare three different MIP formulations of the problem. We prove that one of the formulations is weaker since it may provide worse LP relaxation bounds. A set of instances are randomly generated and extensive computational experiments are conducted to compare these formulations. The results show that one of the formulation gives the overall best performance on both real-world instances and randomly generated instances. A library of instances is available online, and we hope that this can stimulate further research on this very challenging rich real-world LSP (<http://decisionbrain.com/ISCO2016>).

References

1. Absi, N., Kedad-Sidhoum, S., Dautère-Pères, S.: Uncapacitated lot-sizing problem with production time windows, early productions, backlogs and lost sales. *Int. J. Prod. Res.* **49**(9), 2551–2566 (2011)
2. Brahimi, N., Dautère-Pères, S., Wolsey, L.A.: Polyhedral and lagrangian approaches for lot sizing with production time windows and setup times. *Comput. Oper. Res.* **37**(1), 182–188 (2010)
3. Dautère-Pères, S., Brahimi, N., Najid, N., Nordli, A.: The single-item lot sizing problem with time windows. Technical report 02/4/AUTO, École des Mines de Nantes, France (2002)

4. Dillenberger, C., Escudero, L.F., Wollensak, A., Zhang, W.: On solving a large-scale resource allocation problem in production planning. In: Fandel, G., Gullledge, T., Jones, A. (eds.) *Operations Research in Production Planning and Control*, pp. 105–119. Springer, Heidelberg, Berlin (1993)
5. Fiorotto, D.J., Araujo, S.A.D.: Reformulation and a Lagrangian heuristic for lot sizing problem on parallel machines. *Ann. Oper. Res.* **217**(1), 213–231 (2014)
6. Gopalakrishnan, M.: A modified framework for modelling set-up carryover in the capacitated lotsizing problem. *Int. J. Prod. Res.* **38**(14), 3421–3424 (2000)
7. Gören, H.G., Tunal, S.: Solving the capacitated lot sizing problem with setup carryover using a new sequential hybrid approach. *Appl. Intell.* **42**(4), 805–816 (2015)
8. Haase, K.: *Lotsizing and Scheduling for Production Planning*. LNEM, vol. 408. Springer, Berlin (1994)
9. Hwang, H.: Dynamic lot-sizing model with production time windows. *Nav. Res. Log.* **54**(6), 692–701 (2007)
10. Mittelman, H.D., Pruessner, A.: A server for automated performance analysis of benchmarking data. *Optim. Method Softw.* **21**(1), 105–120 (2006)
11. Pochet, Y., Wolsey, L.A.: *Production Planning by Mixed Integer Programming*. Springer, New York (2006)
12. Sandbothe, R.A., Thompson, G.L.: A forward algorithm for the capacitated lot size model with stockouts. *Oper. Res.* **38**(3), 474–486 (1990)
13. Sox, C.R., Gao, Y.: The capacitated lot sizing problem with setup carry-over. *IIE Trans.* **31**(2), 173–181 (1999)
14. Suerie, C., Stadtler, H.: The capacitated lot-sizing problem with linked lot sizes. *Manage. Sci.* **49**(8), 1039–1054 (2003)
15. Toledo, F.M.B.: *Dimensionamento de Lotes em Máquinas Paralelas*. Tese (Doutorado), Faculdade de Engenharia Elétrica e Computação, Universidade Estadual de Campinas, Campinas (1998)
16. Toledo, F.M.B., Armentano, V.A.: A lagrangian-based heuristic for the capacitated lot-sizing problem in parallel machines. *Eur. J. Oper. Res.* **175**(2), 1070–1083 (2006)
17. Wolsey, L.A.: Lot-sizing with production and delivery time windows. *Math. Program.* **107**(3), 471–489 (2006)
18. Zangwill, W.I.: A backlogging model and a multi-echelon model of a dynamic economic lot size production system - a network approach. *Manage. Sci.* **15**(9), 506–527 (1969)